

TP02

Keystone : identité, projets et contrôle d'accès

Module « Infrastructures cloud natives » · 2026-2027

CONTENU DU DOCUMENT

- 01** Objectifs du TP
- 02** Partie 1 — De la CLI aux API REST d'OpenStack
- 03** Partie 2 — Projets, utilisateurs et rôles
- 04** Partie 3 — Isolation et contrôle d'accès

MODULE

Infrastructures cloud natives

PLATEFORME

OpenStack · Keystone

ANNÉE

2026-2027

00 Objectifs du TP

Ce TP porte sur Keystone, le service d'identité d'OpenStack. Vous allez découvrir comment un client s'authentifie et trouve les services du cloud, puis comment Keystone, avec les autres services, isole les projets et contrôle ce que chaque utilisateur peut faire.

À l'issue du TP, vous serez capable de :

Partie 1 — De la CLI aux API REST

1. Charger une identité avec `openrc` et expliquer le rôle des variables `OS_*` .
2. Expliquer ce que sont un jeton, le catalogue de services et un endpoint, et comment ils s'articulent.
3. Lire la sortie de la CLI en mode `--debug` pour reconstituer l'échange HTTP entre le client et les services (découverte de version, authentification, appel métier).
4. Interroger une API OpenStack directement avec `curl` et un jeton, et expliquer l'effet de sa révocation.

Partie 2 — Projets, utilisateurs et rôles

1. Décrire l'organisation de Keystone : domaines, projets, utilisateurs, groupes et rôles.
2. Créer des projets et des utilisateurs, et leur attribuer des rôles.
3. Gérer plusieurs identités avec le fichier `clouds.yaml` .

Partie 3 — Isolation et contrôle d'accès

1. Vérifier l'isolation entre projets (multi-tenancy) : un utilisateur ne voit pas les ressources d'un autre projet.
2. Observer le contrôle d'accès basé sur les rôles (RBAC) : distinguer les droits des rôles `admin` , `member` et `reader` , et interpréter les refus (403) et les ressources introuvables (404).
3. Lire la politique d'accès d'un service pour expliquer pourquoi une opération est autorisée ou refusée.

Partie 1 De la CLI aux API REST d'OpenStack

- La CLI `openstack` n'est qu'un client HTTP : elle demande un jeton à Keystone, lit le catalogue pour trouver l'URL du service, puis appelle son API REST.

1.1. Charger son identité : `source openrc`

- Avant toute commande, la CLI doit savoir qui vous êtes et où se trouve Keystone. DevStack fournit pour cela le script `openrc` :

TERMINAL

```
cd ~/devstack
source openrc admin admin      # syntaxe : source openrc <utilisateur> <projet>
env | grep ^OS_                # affiche les variables chargées
```

Élément	Rôle
<code>source</code>	Exécute le script dans le terminal courant : les variables qu'il définit restent disponibles pour les commandes suivantes. Lancé sans <code>source</code> (<code>./openrc</code>), le script s'exécuterait dans un sous-shell et les variables disparaîtraient à la fin.
<code>openrc</code>	Script de DevStack qui définit les variables d'environnement <code>OS_*</code> .
1er argument (<code>admin</code>)	Nom de l' utilisateur .
2e argument (<code>admin</code>)	Nom du projet auquel le jeton sera lié (<i>scope</i>).

Principales variables chargées :

Variable	Contenu	Utilisée pour
<code>OS_AUTH_URL</code>	<code>http://<IP>/identity</code>	La seule adresse que la CLI connaît au départ : celle de Keystone
<code>OS_USERNAME</code> / <code>OS_PASSWORD</code>	Identifiants de l'utilisateur	Le corps de la requête d'authentification
<code>OS_PROJECT_NAME</code>	Projet visé	Le <i>scope</i> du jeton
<code>OS_USER_DOMAIN_NAME</code> , <code>OS_PROJECT_DOMAIN_NAME</code>	Domaine (<code>Default</code>)	Identifier sans ambiguïté l'utilisateur et le projet
<code>OS_REGION_NAME</code>	<code>RegionOne</code>	Choisir les endpoints de la bonne région dans le catalogue

À retenir

ces variables ne contiennent **pas de jeton**, mais des **identifiants**. La CLI les envoie à Keystone à chaque commande pour obtenir un nouveau jeton. Elles n'existent que dans le terminal où elles ont été chargées : il faut refaire `source openrc` dans chaque nouveau terminal ou nouvelle connexion SSH.

1.2. Jeton (token), catalogue et endpoints

1.2.1. Jeton (token)

- Lancez la commande ci-dessous :

TERMINAL

```
openstack token issue
```

Commande	Ce qu'elle fait	Service interrogé	Ce qu'elle affiche	À retenir
<code>openstack token issue</code>	Demande un nouveau jeton à Keystone avec les identifiants chargés (<code>openrc</code> ou <code>clouds.yaml</code>) et l'affiche en clair.	Keystone	<code>expires</code> : date et heure d'expiration ; <code>id</code> : le jeton lui-même ; <code>project_id</code> : projet auquel le jeton est lié (<code>scope</code>) ; <code>user_id</code> : utilisateur authentifié.	Le jeton est une preuve d'identité temporaire : par défaut, Keystone le rend valable 1 heure. Il est lié à un seul projet : changer de projet impose un nouveau jeton.

Q1

Quelle est la durée de validité du jeton ? À quel projet est-il lié (`scope`) ?

Note : La commande source `openrc` ne charge pas de jeton, elle charge des identifiants (nom d'utilisateur, mot de passe, projet, URL de Keystone) dans des variables d'environnement `OS_*`. Ces variables n'expirent pas. la CLI s'en sert pour demander un nouveau jeton à chaque commande. L'expiration au bout d'une heure ne la gêne donc jamais.

Pourquoi cette commande ?

- Chaque commande `openstack` (ex. `openstack server list`) demande automatiquement un **nouveau jeton** à Keystone, l'utilise, puis le jette, sans vous le montrer.
- `openstack token issue` fait uniquement la première étape : elle **demande un jeton et l'affiche**. Elle sert à :

- Vérifier l'authentification : si elle échoue, le problème vient de Keystone ou de vos variables `OS_*` (vérifier les valeurs).
- Observer le contenu d'un jeton : utilisateur (`user_id`), projet (`project_id`) et date d'expiration (`expires`).
- Réutiliser le jeton pour appeler directement les API REST :

À retenir

Toute requête vers un service OpenStack passe d'abord par Keystone pour obtenir un jeton. Le jeton est ensuite envoyé dans l'en-tête HTTP `X-Auth-Token`.

1.2.2. Catalogue

- Lancez la commande ci-dessous :

TERMINAL

```
openstack catalog list
```

Commande	Ce qu'elle fait	Service interrogé	Ce qu'elle affiche	À retenir
<code>openstack catalog list</code>	Affiche le catalogue de services que Keystone renvoie avec chaque jeton.	Keystone (le catalogue fait partie de la réponse d'authentification)	Name : nom du service (<code>nova</code> , <code>glance</code> ...) ; Type : type normalisé (<code>compute</code> , <code>image</code> ...) ; Endpoints : pour chaque région et interface, l'URL du service.	C'est l' annuaire du cloud : la CLI, Horizon ou Terraform y cherchent l'URL d'un service par son type . Il dépend du jeton : un utilisateur ne voit que les services auxquels son projet a accès.

1.2.3. Endpoints

- Lancez la commande ci-dessous :

TERMINAL

```
openstack endpoint list --interface public
```

Commande	Ce qu'elle fait	Service interrogé	Ce qu'elle affiche	À retenir
<code>openstack endpoint list --interface public</code>	Liste les endpoints enregistrés dans Keystone, filtrés sur l'interface <code>public</code> .	Keystone	ID , Region , Service Name , Service Type , Enabled , Interface , URL : une ligne par endpoint.	Même information que le catalogue, mais vue côté administration : une ligne par endpoint, avec son ID, ce qui permet de le modifier ou le supprimer. L'interface <code>public</code> sert aux utilisateurs ; <code>internal</code> sert aux échanges entre services ; <code>admin</code> sert aux opérations d'administration. Cette commande demande en général le rôle <code>admin</code> .

Q2

Comparez les sorties de `openstack catalog list` et de `openstack endpoint list --interface public` :

a) Les URL sont-elles les mêmes dans les deux sorties ? Relevez le cas du service `nova_legacy` : quelle différence observez-vous entre les deux commandes, et comment l'expliquez-vous ?

b) Tous les services utilisent la même adresse IP. Comment le serveur sait-il à quel service transmettre une requête ?

c) Relancez `openstack endpoint list` **sans** l'option `--interface public`. Le résultat change-t-il ? Quelles interfaces DevStack a-t-il déclarées ? Pourquoi un cloud de production déclarerait-il aussi une interface `internal` ?

Dans Keystone, un même service peut être enregistré avec plusieurs adresses, une par interface, c'est-à-dire selon qui l'appelle et depuis quel réseau

d) Changez d'identité en chargeant l'utilisateur `demo` du projet `demo`, puis relancez les trois commandes :

TERMINAL

```
source ~/devstack/openrc demo demo
openstack token issue
openstack catalog list
openstack endpoint list --interface public
```

Comparez avec les résultats obtenus en tant qu'admin : qu'est-ce qui change dans le jeton ? Dans l'URL de `nova_legacy` ? Que se passe-t-il pour `endpoint list`, et pourquoi ?

Revenez ensuite à l'identité admin : `source ~/devstack/openrc admin admin`.

1.3. Observer une commande en mode debug

Clés de lecture : les requêtes HTTP en mode `--debug`

Une commande `openstack` envoie en réalité plusieurs **requêtes HTTP** aux services.

1. Une requête (**REQ:**) se lit comme une commande `curl` :

```
REQ: curl -g -i -X GET http://<IP>/compute/v2.1/servers/detail -H "X-Auth-Token: {SHA256}3f9a..."
```

Élément	Signification
<code>-X GET</code>	Méthode : <code>GET</code> lire · <code>POST</code> créer/envoyer · <code>PUT</code> modifier · <code>DELETE</code> supprimer
<code>/compute</code>	Service contacté (<code>/identity</code> Keystone · <code>/compute</code> Nova · <code>/image</code> Glance)
<code>/v2.1/servers/detail</code>	Version de l'API, puis ressource demandée
<code>-H "..."</code>	En-têtes : informations jointes à la requête (format, jeton...)

2. Une réponse (**RESP:**) commence par un code de statut :

`200` OK · `201` créé · `300` plusieurs choix (ex. plusieurs versions d'API) · `401` non authentifié · `403` non autorisé

3. L'authentification est affichée à part. Pour ne pas exposer votre mot de passe, la requête (un `POST` vers Keystone) n'apparaît pas sous forme `REQ: curl`, mais sous la forme `Making authentication request to .../v3/auth/tokens`.

4. Reconnaître le type de requête :

- *Authentification* : ligne `Making authentication request`
- *Découverte de version* : l'URL s'arrête au service ou à sa version (`/identity`, `/compute/v2.1`)
- *Requête métier* : une ressource suit la version (`/servers`, `/images` ...)

Q3

Lancez la commande ci-dessous, puis analysez sa sortie à l'aide des clés de lecture ci-dessus. (l'option `--debug` pour afficher les requêtes sur la sortie d'erreur ; `2>&1` pour la rediriger pour pouvoir la filtrer avec `grep`)

TERMINAL

```
openstack --debug server list 2>&1 | grep -iE 'REQ:|RESP:|authentication request'
```

a) Combien d'échanges HTTP observez-vous (en comptant l'authentification) ? Pour chacun, indiquez la méthode, le service contacté, le code de réponse et le type de requête.

b) Pourquoi Keystone est-il toujours contacté en premier ?

c) Où la CLI trouve-t-elle l'URL des services contactés après Keystone ?

1.4. Interroger l'API sans la CLI

La CLI n'est qu'un client HTTP. Vous allez faire vous-même ses deux étapes : obtenir un jeton, puis appeler un service.

Vérifiez que vous utilisez l'identité admin

```
source ~/devstack/openrc admin admin
```

1.4.1. Jeton et URL de Nova (récupérés une fois avec la CLI)

TERMINAL

```
export TOKEN=$(openstack token issue -f value -c id)
export NOVA=$(openstack endpoint list --service compute --interface public -f value -c URL)
```

Élément	Rôle
<code>export TOKEN=...</code>	Crée la variable d'environnement <code>TOKEN</code> , réutilisable ensuite avec <code>\$TOKEN</code> .
<code>\$(...)</code>	Exécute la commande entre parenthèses et remplace l'expression par son résultat.
<code>openstack token issue</code>	Demande un nouveau jeton à Keystone.
<code>-f value</code>	<i>Format</i> : affiche les valeurs brutes, sans tableau ni bordures. Autres formats : <code>table</code> (par défaut), <code>json</code> , <code>yaml</code> , <code>shell</code> .
<code>-c id</code>	<i>Column</i> : ne garde que la colonne <code>id</code> , qui contient le jeton.
<code>openstack endpoint list --service compute --interface public</code>	Cherche dans Keystone l'URL publique du service <code>compute</code> (Nova), stockée dans <code>NOVA</code> .

1.4.2. Appel direct à l'API de Nova

TERMINAL

```
curl -s -H "X-Auth-Token: $TOKEN" $NOVA/servers
curl -si $NOVA/servers | head -1
```

Élément	Commande 1 : avec jeton	Commande 2 : sans jeton
<code>curl</code>	Envoie une requête HTTP <code>GET</code> à l'URL indiquée.	Idem.
<code>-s</code>	<i>Silent</i> : masque la barre de progression.	Idem.

Élément	Commande 1 : avec jeton	Commande 2 : sans jeton
<code>-i</code>	—	<i>Include</i> : affiche aussi les en-têtes de la réponse, dont la ligne de statut HTTP.
<code>-H "X-Auth-Token: \$TOKEN"</code>	Ajoute l'en-tête qui porte le jeton : la preuve d'identité.	Absent : aucune preuve d'identité n'est envoyée.
<code>\$NOVA/servers</code>	Ressource « liste des instances » de l'API Nova.	Idem.
<code>\ head -1</code>	—	Ne garde que la première ligne de la réponse : le code HTTP.
Résultat	<code>{"servers": []}</code> : Nova accepte la requête et renvoie la liste.	<code>HTTP/1.1 401 Unauthorized</code> : Nova refuse, faute de jeton.

Une seule commande CLI, `openstack server list`, enchaîne automatiquement plusieurs requêtes HTTP : négociation de version, authentification auprès de Keystone, recherche du service dans le catalogue, puis appel à Nova. Elle met ensuite la réponse en forme. Avec `curl`, chacune de ces étapes doit être faite à la main.

La CLI est une abstraction des API REST.

1.4.3. Révocation d'un jeton

TERMINAL

```
export TOKEN2=$(openstack token issue -f value -c id)
openstack token revoke $TOKEN2
curl -s -H "X-Auth-Token: $TOKEN2" $NOVA/servers
curl -s -H "X-Auth-Token: $TOKEN" $NOVA/servers
openstack server list
```

Élément	Rôle
<code>export TOKEN2=...</code>	Obtient un second jeton , indépendant du premier.
<code>openstack token revoke \$TOKEN2</code>	Demande à Keystone d'invalider ce jeton. N'affiche rien en cas de succès.
<code>curl ... \$TOKEN2 ...</code>	Appelle Nova avec le jeton révoqué → <code>HTTP/1.1 401 Unauthorized</code> : Nova fait valider le jeton par Keystone, qui le déclare révoqué.
<code>curl ... \$TOKEN ...</code>	Même appel avec le premier jeton → <code>HTTP/1.1 200 OK</code> : seul le jeton révoqué est refusé.
<code>openstack server list</code>	Fonctionne normalement : la CLI n'utilise ni <code>\$TOKEN</code> ni <code>\$TOKEN2</code> , elle obtient un nouveau jeton avec les identifiants d' <code>openrc</code> .

À retenir

Le jeton est la preuve d'identité obligatoire pour toute requête métier. Il est délivré par Keystone, présenté dans l'en-tête `X-Auth-Token`, et vérifié par le service appelé auprès de Keystone. Sans jeton valide, aucun service n'exécute de requête, même une simple lecture.

- Un jeton a une durée de vie limitée (expiration) et peut être invalidé à tout moment (révocation). Il ne contient pas le mot de passe : on peut couper un accès sans changer les identifiants de l'utilisateur.

- Révoquer un jeton n'affecte que ce jeton : les autres jetons du même utilisateur restent valides.

Partie 2 Projets, utilisateurs et rôles

- Vous allez créer deux équipes et trois utilisateurs.

Utilisateur	Projet	Rôle	Usage dans le TP
candace	team-a	member	Opère les ressources de l'équipe A
baron	team-b	member	Opère les ressources de l'équipe B
ian	team-a	reader	Consulte l'équipe A sans rien modifier

2.1 Explorer l'existant

TERMINAL

```
openstack domain list
openstack project list
openstack user list
openstack role list
openstack role assignment list --names
```

Q4

Quels projets et utilisateurs DevStack a-t-il créés ? Quels rôles existent ? Que signifie la colonne **Inherited** ?

2.2 Créer projets, utilisateurs et attributions

- Étape 1 : créer les projets

TERMINAL

```
openstack project create --domain Default --description "Équipe A" team-a
openstack project create --domain Default --description "Équipe B" team-b
```

- Étape 2 : créer les utilisateurs, sans rôle

TERMINAL

```
openstack user create --domain Default --project team-a --password 'Candace#2026' candace
openstack user create --domain Default --project team-b --password 'Baron#2026' baron
openstack user create --domain Default --project team-a --password 'Ian#2026' ian
```

- Étape 3 : tenter de s'authentifier en tant que candace

TERMINAL

```
openstack --os-username candace --os-password 'Candace#2026' --os-project-name team-a token issue
```

- Étape 4 : attribuer les rôles

TERMINAL

```
openstack role add --project team-a --user candace member
openstack role add --project team-b --user baron member
openstack role add --project team-a --user ian reader
```

- Étape 5 : tenter de s'authentifier en tant que candace (2ème tentative)

TERMINAL

```
openstack --os-username candace --os-password 'Candace#2026' --os-project-name team-a token issue
```

Q5

L'option `--project` de `user create` donne-t-elle un rôle à l'utilisateur ?

- Pour qu'un utilisateur puisse travailler dans un projet, il faut une attribution de rôle **explicite** (`role add`) : le triplet utilisateur · rôle · projet. Sans elle, Keystone refuse même de délivrer un jeton lié à ce projet, puisque l'utilisateur n'y a aucun rôle.

2.3 Un profil par identité avec clouds.yaml

- Avec `source openrc`, on ne peut charger qu'une identité à la fois, et il faut recharger le script pour en changer.
- Le fichier `clouds.yaml` règle ce problème : c'est un **carnet de connexions**, où chaque **profil** regroupe tout ce qu'il faut pour se connecter sous une identité. On choisit ensuite le profil par son nom.
- Ce fichier est le format standard des clients OpenStack (CLI, SDK Python, Ansible, Terraform).

	<code>source openrc</code>	<code>clouds.yaml</code>
Identités	Une seule à la fois	Autant de profils que nécessaire
Changer d'identité	Recharger le script	<code>--os-cloud <profil></code> (une commande) ou <code>export OS_CLOUD=<profil></code> (tout le terminal)
Durée de vie	Le temps du terminal	Permanent (fichier sur le disque)

Prérequis

les utilisateurs candace, baron et ian doivent avoir été créés en 2.2, avec les mots de passe ci-dessous et un rôle sur leur projet.

- Créez le fichier `clouds.yaml` dans `~/.config/openstack` :

TERMINAL

```
mkdir -p ~/.config/openstack
nano ~/.config/openstack/clouds.yaml
```

- Copiez-collez le contenu ci-dessous. Remplacez `<ADMIN_PASSWORD>` par le mot de passe de l'utilisateur admin, déclaré dans le fichier `local.conf`. Enregistrez avec **Ctrl + O** puis **Entrée**, et quittez avec **Ctrl + X**.

TERMINAL

```
clouds:
  admin:
    auth:
      auth_url: http://localhost/identity
      username: admin
      password: '<ADMIN_PASSWORD>'
      project_name: admin
      user_domain_name: Default
      project_domain_name: Default
      identity_api_version: 3
  candace:
    auth:
      auth_url: http://localhost/identity
      username: candace
      password: 'Candace#2026'
      project_name: team-a
      user_domain_name: Default
      project_domain_name: Default
      identity_api_version: 3
  baron:
    auth:
      auth_url: http://localhost/identity
      username: baron
      password: 'Baron#2026'
      project_name: team-b
      user_domain_name: Default
      project_domain_name: Default
      identity_api_version: 3
  ian:
    auth:
      auth_url: http://localhost/identity
      username: ian
      password: 'Ian#2026'
      project_name: team-a
      user_domain_name: Default
      project_domain_name: Default
      identity_api_version: 3
```

Attention

gardez les apostrophes autour des mots de passe. En YAML, `#` commence un commentaire : sans apostrophes, `Candace#2026` serait lu `Candace` .

- Modifiez les permissions d'accès du fichier pour qu'il ne soit lisible et modifiable que par son propriétaire, car il contient des mots de passe en clair :

TERMINAL

```
chmod 600 ~/.config/openstack/clouds.yaml
```

- Videz les variables `OS_*` chargées par `openrc` (dans le terminal courant, ou dans une seconde connexion SSH à votre instance) : sinon, elles prennent le pas sur `clouds.yaml` .

TERMINAL

```
unset $(env | grep '^OS_' | cut -d= -f1)
```

- Testez le chargement de chaque profil :

TERMINAL

```
openstack --os-cloud admin token issue
openstack --os-cloud candace token issue
openstack --os-cloud baron token issue
openstack --os-cloud ian token issue
```

Q6

Pour chaque jeton, relevez le projet et l'identifiant utilisateur. Quels profils partagent le même projet ? Qu'est-ce que cela signifie ?

- Pour la suite, choisissez l'identité de tout le terminal avec `export OS_CLOUD=<profil>` : les commandes suivantes s'exécutent alors sous ce profil, sans option. Inutile de demander un jeton au préalable : chaque commande s'authentifie elle-même.

2.4 Groupes

- Créez un groupe `ops`, ajoutez-y candace, puis donnez au groupe le rôle `reader` sur team-b.

TERMINAL

```
openstack group create ops
openstack group add user ops candace
openstack role add --group ops --project team-b reader
openstack role assignment list --names --effective --user candace
```

- Listez tous les rôles de candace :

TERMINAL

```
openstack role assignment list --names --user candace
```

- Listez tous les rôles **effectifs** de candace : ceux qui lui sont attribués directement et ceux qu'elle obtient par son appartenance à un groupe :

TERMINAL

```
openstack role assignment list --names --effective --user candace
```

Q7

Pourquoi l'option `--effective` change-t-elle le résultat ? Quel est l'intérêt des groupes quand 40 étudiants rejoignent un projet ?

- Retirez ensuite le rôle du groupe, pour ne pas fausser la partie 3, puis vérifiez :

TERMINAL

```
openstack --os-cloud admin role remove --group ops --project team-b reader
openstack --os-cloud admin role assignment list --names --effective --user candace
```

Seul le rôle `member` sur team-a doit apparaître.

Partie 3 Isolation et contrôle d'accès

Chaque projet est un **locataire** (*tenant*) isolé : ses réseaux, VM et groupes de sécurité appartiennent au projet, et non aux utilisateurs.

Le modèle RBAC

OpenStack applique un contrôle d'accès basé sur les rôles (*Role-Based Access Control*, RBAC) : on n'attribue pas des droits directement à un utilisateur, mais un **rôle** sur un **projet**. Deux contrôles s'enchaînent à chaque requête :

- **Le projet** du jeton détermine ce que l'utilisateur **voit** : sans rôle sur un projet, ses ressources sont invisibles (404). Seules les ressources explicitement partagées, comme le réseau **public**, sont visibles par tous.
- **Le rôle** détermine ce qu'il peut **faire** : un **reader** lit, un **member** crée et modifie, un **admin** administre. Une action non autorisée par le rôle est refusée (403).

Pour chaque test, notez la commande, le résultat attendu, le résultat obtenu et, en cas d'erreur, le code HTTP.

L'identité utilisée est indiquée au début de chaque bloc (**[admin]**, **[candace]**, **[baron]**, **[ian]**). Choisissez-la avec `export OS_CLOUD=<profil>`, ou ajoutez `--os-cloud <profil>` à la commande.

3.1 Candace déploie l'infrastructure de l'équipe A

[candace]

TERMINAL

```
export OS_CLOUD=candace
openstack network create net-candace
openstack subnet create --network net-candace --subnet-range 10.10.1.0/24 --dns-nameserver 8.8.8.8 subnet-candace
openstack router create --external-gateway public router-candace
openstack router add subnet router-a subnet-candace
openstack security group rule create --protocol icmp default
openstack security group rule create --protocol tcp --dst-port 22 default

IMG=$(openstack image list -f value -c Name | grep -i cirros | head -1)
openstack server create --flavor m1.tiny --image "$IMG" --network net-a --wait vm-candace
openstack server list
```

3.2 Baron tente d'accéder aux ressources de l'équipe A

Notez d'abord l'ID de **vm-candace** et de **net-candace** (affichés par candace) et remplacez le dans les commandes suivantes.

[baron]

Notez d'abord l'ID de **vm-candace** et de **net-candace** (affichés par candace) et remplacez le dans les commandes suivantes.

TERMINAL

```
export OS_CLOUD=baron
openstack server list
openstack network list
openstack server show <ID_vm-candace>
openstack server delete <ID_vm-candace>
openstack network show <ID_net-candace>
openstack server create --flavor m1.tiny --image "$IMG" --network <ID_net-candace> vm-intrus
```

Q8

Baron reçoit-il « accès refusé » (403) ou « introuvable » (404) ? Pourquoi est-il préférable, du point de vue sécurité, de répondre 404 ?

Q9

Quel réseau Baron voit-il alors qu'il ne l'a pas créé ? Quelle propriété de ce réseau l'explique (`openstack network show public`) ?

3.3 Ian, lecteur de l'équipe A

[ian]

TERMINAL

```
export OS_CLOUD=ian
openstack server list
openstack server show vm-candace
openstack server stop vm-candace
openstack server create --flavor m1.tiny --image "$IMG" --network net-candace vm-ian
openstack network create net-ian
```

Q10

Quelles actions réussissent pour ian, et lesquelles échouent ? Relevez le message d'erreur obtenu. Comparez avec le code obtenu par baron en Q8 : pourquoi ian reçoit-il un refus (403) et non « introuvable » (404) ?

3.4 Ce qu'un membre ne peut pas faire

[candace]

TERMINAL

```
export OS_CLOUD=candace
openstack user list
openstack project create team-c
openstack flavor create --vcpus 8 --ram 16384 --disk 100 m1.geant
openstack quota set --instances 100 team-a
```

Q11

Complétez le tableau à partir de vos tests. Pour une case non encore testée, lancez la commande indiquée avec l'identité correspondante (`--os-cloud <profil>`).

Préparation : récupérez le nom de l'image CirrOS et les identifiants de `vm-candace` et `net-candace` . Les commandes du tableau utilisent les **ID** : une ressource d'un autre projet ne peut pas être retrouvée par son nom.

TERMINAL

```
IMG=$(openstack --os-cloud admin image list -f value -c Name | grep -i cirros | head -1)
VM_ID=$(openstack --os-cloud candace server show vm-candace -f value -c id)
NET_ID=$(openstack --os-cloud candace network show net-candace -f value -c id)
echo "$IMG | $VM_ID | $NET_ID"
```

Légende : ✓ = autorisé · ✗ 403 = refusé (rôle insuffisant) · ✗ 404 = introuvable (ressource invisible)

Dans les deux dernières colonnes, indiquez **Oui** si le résultat varie selon le **projet** de l'utilisateur (apparition de 404), ou selon son **rôle** (apparition de 403). Une action peut dépendre des deux.

`<profil>` désigne l'identité testée : admin, candace, baron ou ian.

#	Action	Commande	admin	candace	baron	ian	Dépend du projet ?	Dépend du rôle ?
1	Voir <code>vm-candace</code>	<code>openstack server show \$VM_ID</code>						
2	Créer une VM dans son projet	<code>openstack server create --flavor ml.tiny --image "\$IMG" --network shared --wait vm-test-<profil></code>						
3	Arrêter <code>vm-candace</code>	<code>openstack server stop \$VM_ID</code>						
4	Créer un réseau dans son projet	<code>openstack network create net-test-<profil></code>						
5	Voir le réseau <code>net-candace</code>	<code>openstack network show \$NET_ID</code>						
6	Lister les utilisateurs	<code>openstack user list</code>						
7	Créer un flavor	<code>openstack flavor create --vcpus 1 --ram 256 --disk 1 test-flavor</code>						

Ligne 2 : le réseau `shared`, créé par DevStack, est visible par tous les projets. Il permet de tester la création d'une VM avec chaque identité, sans réseau propre à chaque projet.

Ligne 3 : après chaque arrêt réussi, redémarrez la VM avec `openstack --os-cloud candace server start $VM_ID`. Sinon, le test suivant échoue avec une erreur `409 Conflict` (VM déjà arrêtée), sans rapport avec les droits.

Q12

À partir du tableau, expliquez pourquoi baron obtient des 404, et ian des 403.

Nettoyage : chaque identité supprime les ressources de test qu'elle a créées. Conservez `vm-candace` et `net-candace`, qui servent dans la suite.

TERMINAL

```
openstack --os-cloud admin server delete vm-test-admin
openstack --os-cloud candace server delete vm-test-candace
openstack --os-cloud baron server delete vm-test-baron

openstack --os-cloud admin network delete net-test-admin
openstack --os-cloud candace network delete net-test-candace
openstack --os-cloud baron network delete net-test-baron

openstack --os-cloud admin flavor delete test-flavor
```

3.5 Le cas d'admin : voir tous les projets

- Admin a le rôle `admin`, qui lui permet d'administrer tout le cloud. Voit-il pour autant les ressources de tous les projets ?

[admin] Lancez ces deux commandes et comparez :

TERMINAL

```
openstack --os-cloud admin server list
openstack --os-cloud admin server list --all-projects
```

Affichez ensuite le projet auquel le jeton d'admin est lié :

TERMINAL

```
openstack --os-cloud admin token issue -c project_id
openstack --os-cloud admin project list
```

Q13

`vm-candace` apparaît-elle dans les deux listes ? À quel projet le jeton d'admin est-il lié ? Expliquez pourquoi admin ne voit pas `vm-candace` sans `--all-projects`, alors qu'il a le rôle `admin`.

3.6. Où sont définies les règles d'accès ?

- Vous avez constaté que candace peut créer une VM et ian non, et que seul admin peut utiliser `--all-projects`. Ces décisions ne sont pas prises par Keystone : Keystone dit seulement **qui** vous êtes et **quels rôles** vous avez.
- C'est **chaque service** (Nova, Neutron, Glance...) qui décide ensuite si vos rôles autorisent l'opération demandée, en consultant sa **politique d'accès** (*policy*).

Notion	Signification
Politique d'accès (<i>policy</i>)	Ensemble de règles propre à chaque service : pour chaque opération de l'API, la condition à remplir
Règle	Une ligne de la forme <code>"nom": "condition"</code>
Règles par défaut	Écrites dans le code du service ; elles s'appliquent tant que l'administrateur ne les modifie pas
<code>policy.yaml</code>	Fichier (par exemple <code>/etc/nova/policy.yaml</code>) où l'administrateur peut remplacer certaines règles

Étape 1 : afficher les règles de deux opérations testées

L'outil `oslopolicy-sample-generator` affiche toutes les règles par défaut d'un service. Il est installé dans l'environnement Python de DevStack : on l'appelle par son chemin complet.

TERMINAL

```
POLICY=/opt/stack/data/venv/bin/oslopolicy-sample-generator

# Créer une VM
$POLICY --namespace nova | grep 'servers:create'

# Lister les VM de tous les projets (--all-projects)
$POLICY --namespace nova | grep all_tenants
```

Clés de lecture d'une règle

Élément	Signification
# en début de ligne	Valeur par défaut, active tant qu'un <code>policy.yaml</code> ne la remplace pas
<code>rule:<nom></code>	Renvoi : « appliquer la règle nommée <code><nom></code> », définie ailleurs dans la politique
<code>role:<rôle></code>	L'utilisateur doit avoir ce rôle
<code>project_id:%(project_id)s</code>	Le projet du jeton de l'utilisateur doit être celui de la ressource visée
<code>and</code> / <code>or</code>	Les deux conditions / l'une des deux

Étape 2 : décomposer la règle de création d'une VM

À l'étape 1, la règle de création d'une VM affiche :

POLITIQUE

```
#"os_compute_api:servers:create": "rule:project_member_or_admin"
```

Elle ne contient aucun rôle : elle renvoie à une autre règle, `project_member_or_admin`. Pour savoir ce qu'elle exige vraiment, il faut afficher cette règle :

TERMINAL

```
$POLICY --namespace nova | grep '^#"project_member_or_admin"'
```

Cette règle renvoie elle-même à deux autres règles. Affichez-les à leur tour :

TERMINAL

```
$POLICY --namespace nova | grep -E '^#"(project_member_api|context_is_admin)"'
```

Vous obtenez cette fois des conditions de base (`role:...`, `project_id:...`) : la décomposition est terminée.

Q14

a) Représentez la règle de création d'une VM sous forme d'arbre, de `os_compute_api:servers:create` jusqu'aux conditions de base. Traduisez-la en une phrase en français.

POLITIQUE

```
os_compute_api:servers:create
├─ rule:
│   └─ rule: ---
│       └─ role: ---
│   OU rule: ---
│       └─ role: ---
```

b) À l'aide de votre arbre, complétez le tableau (Oui / Non), puis expliquez pourquoi candace peut créer une VM dans team-a, alors que ian (même projet, rôle `reader`) et baron (rôle `member`, mais projet team-b) ne le peuvent pas.

Utilisateur	A le rôle <code>member</code> ?	Son jeton est-il lié au projet team-a ?	A le rôle <code>admin</code> ?	Création autorisée ?
candace				

Utilisateur	A le rôle <code>member</code> ?	Son jeton est-il lié au projet team-a ?	A le rôle <code>admin</code> ?	Création autorisée ?
ian				
baron				

Pour chaque refus, indiquez **quelle condition** de l'arbre n'est pas remplie.

c) À l'étape 1, quelle règle protège `--all-projects` ? Expliquez pourquoi seul admin peut l'utiliser.